

주문 앱 Entity 설계서.

자세한 기능 다이어그램 : <https://drive.google.com/file/d/1f37XDOy0nTJsDkhAivOzNONkMNr1psgp/view?usp=sharing>

설계 목적 및 원칙

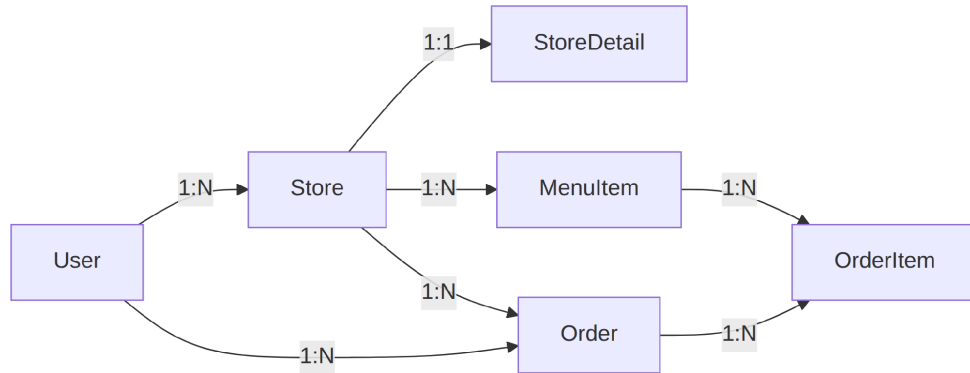
설계 목적 — 이 Entity 설계는 파트 A/B/C가 각자 독립적으로 개발할 수 있도록 테이블 구조와 소유 관계를 명확히 하고, 세 파트가 함께 참조할 공통 기준을 제공하기 위한 것입니다.

- 단일 책임: 가게의 기본 정보(Store)와 영업시간(StoreDetail)을 분리합니다. 영업시간은 조회 빈도와 변경 빈도가 달라 별도 테이블로 관리하는 편이 낫습니다.
- 상태는 Enum으로 관리하되 DB 컬럼은 VARCHAR로 통일:
주문 상태("PENDING/ACCEPTED/READY/COMPLETED/REJECTED/CANCELLED")는 DB 컬럼 타입을 VARCHAR로 두고 자바 Enum + @Enumerated(EnumType.STRING)으로 관리합니다. 상태값이 추가·변경돼도 ALTER TABLE 없이 코드 배포만으로 반영할 수 있습니다.
- 연관관계 최소화: 모든 연관관계는 FetchType.LAZY를 기본으로 하고, 정말 필요한 경우에만 Fetch Join으로 즉시 로딩합니다.
- 장바구니(N:M)는 연결 엔티티로 해소: 하나의 주문에 여러 메뉴가 담기는 구조이므로 Order와 MenuItem을 JPA의 @ManyToMany로 직접 연결하지 않고, OrderItem이라는 연결 엔티티로 풀어냅니다. 주문 시점의 메뉴 이름·가격 스냅샷도 여기에 함께 저장합니다.

Entity 클래스 상세 설계

- **User** — 손님/사장님 통합 계정. 이메일, 비밀번호, 이름, 전화번호, role(USER/OWNER)을 가집니다. 로그인 API 하나로 두 역할을 모두 처리하고, JWT의 role 클레임으로 화면·API 접근을 구분합니다.
- **Store** — 가게. owner(User) 참조, 이름, 주소, 카테고리. 메뉴·주문 이력이 이 가게를 참조하므로 하드 삭제 대신 deleted_at을 채우는 소프트 삭제로 처리합니다.
- **StoreDetail** — 가게 상세(영업시간). Store와 1:1로 연결되며 오픈/마감 시각을 가집니다. 가게 기본 정보와 분리해 관리합니다.
- **MenuItem** — 가게의 메뉴. 이름, 가격, 품절 여부(sold_out), 가게 참조. 주문 이력이 참조하므로 하드 삭제 대신 소프트 삭제(deleted_at)를 씁니다.
- **Order**
주문 마스터. 손님·가게 참조, 상태(PENDING/ACCEPTED/READY/COMPLETED/REJECTED/CANCELLED), 픽업 예정 시각, 주문 시점 총액 스냅샷(total_price). 파트 B가 생성 로직을, 파트 C가 상태 변경·집계 로직을 담당하는 공유 엔티티입니다.
- **OrderItem** — 주문 상세. 주문 참조, 메뉴 참조, 수량, 주문 시점 단가 스냅샷(price). 장바구니에 담긴 메뉴별 항목을 하나씩 기록합니다.

연관관계 매핑 정의



Order와 OrderItem의 관계가 이 설계의 핵심입니다. 손님이 장바구니에 메뉴 여러 개를 담아 주문하면, Order 한 건과 그 아래 OrderItem 여러 건이 함께 생성됩니다. OrderItem에는 주문 시점의 메뉴 이름·가격을 스냅샷으로 저장해서, 나중에 가게가 메뉴 가격을 바꿔도 과거 주문 내역의 금액은 그대로 유지됩니다. Order.total_price는 OrderItem들의 ($\text{price} \times \text{quantity}$) 합계를 주문 생성 시점에 계산해 저장한 값입니다.

연관관계 매핑 전략

- Fetch 전략: 모든 연관관계는 FetchType.LAZY가 기본입니다. 목록 조회처럼 연관 엔티티가 필요 없는 API에서 불필요한 조인이 발생하지 않도록 하기 위함입니다.
- 연관관계의 주인: 외래 키를 실제로 가지는 쪽(예: Order.store_id, OrderItem.order_id)이 항상 연관관계의 주인이 되고, 반대쪽에서 필요하면 mappedBy로 조회만 합니다.
- Cascade 사용 기준: Order가 저장될 때 그 안의 OrderItem들도 함께 저장되어야 하므로 Order → OrderItem에는 CascadeType.ALL을 사용합니다. 반면 Store가 삭제된다고 지난 Order 이력까지 삭제되면 안 되므로 Store → Order에는 Cascade를 걸지 않습니다.
- 소프트 삭제 정책: Store, MenuItem처럼 주문 이력이 참조하는 엔티티는 하드 삭제 대신 deleted_at을 채우는 소프트 삭제를 씁니다.
- 상태값 컬럼 타입: DB 컬럼은 VARCHAR로 통일하고 자바 Enum + @Enumerated(EnumType.STRING)로 관리합니다.

테이블 스키마 설계

users

컬럼명	타입	제약조건	설명
id	BIGINT	PK	
email	VARCHAR(255)	UNIQUE, NOT NULL	
password	VARCHAR(255)	NOT NULL	BCrypt 해시
name	VARCHAR(50)	NOT NULL	
phone	VARCHAR(20)	NULL	
role	VARCHAR(20)	NOT NULL	USER / OWNER / ADMIN
created_at	DATETIME	NOT NULL	
updated_at	DATETIME	NOT NULL	
deleted_at	DATETIME	nullable	소프트 삭제 시간

stores

컬럼명	타입	제약조건	설명
id	BIGINT	PK	
owner_id	BIGINT	FK → users.id, NOT NULL	
name	VARCHAR(100)	NOT NULL	
address	VARCHAR(255)	NOT NULL	
category	VARCHAR(50)	NOT NULL	
image_url	VARCHAR(500)	NULL	매장 대표 이미지
deleted_at	DATETIME	NULL	소프트 삭제. 메뉴·주문 이력이 참조하므로 하드 삭제하지 않음
created_at	DATETIME	NOT NULL	
updated_at	DATETIME	NOT NULL	

store_details

컬럼명	타입	제약조건	설명
id	BIGINT	PK	
store_id	BIGINT	FK → stores.id, UNIQUE, NOT NULL	
open_time	TIME	NULL	
close_time	TIME	NULL	
created_at	DATETIME	NOT NULL	
updated_at	DATETIME	NOT NULL	

menu_items

컬럼명	타입	제약조건	설명
id	BIGINT	PK	

컬럼명	타입	제약조건	설명
store id	BIGINT	FK → stores.id, NOT NULL	
name	VARCHAR(100)	NOT NULL	
price	INT	NOT NULL	
sold out	BOOLEAN	NOT NULL, DEFAULT false	
image url	VARCHAR(500)	NULL	메뉴 이미지
deleted_at	DATETIME	NULL	소프트 삭제. 주문 이력이 참조하므로 하드 삭제하지 않음
created at	DATETIME	NOT NULL	
updated at	DATETIME	NOT NULL	

orders

컬럼명	타입	제약조건	설명
id	BIGINT	PK	
user id	BIGINT	FK → users.id, NOT NULL	
store id	BIGINT	FK → stores.id, NOT NULL	
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	PENDING / ACCEPTED / READY / COMPLETED / REJECTED / CANCELLED
pickup time	DATETIME	NOT NULL	픽업 예정 시각
request notes	VARCHAR(255)	NULL	손님 요청 사항
total price	INT	NOT NULL	주문 시점 총액 스냅샷
created at	DATETIME	NOT NULL	
updated at	DATETIME	NOT NULL	

order_items

컬럼명	타입	제약조건	설명
id	BIGINT	PK	
order id	BIGINT	FK → orders.id, NOT NULL	
menu item id	BIGINT	FK → menu_items.id, NOT NULL	
quantity	INT	NOT NULL	
price	INT	NOT NULL	주문 시점 단가 스냅샷
created at	DATETIME	NOT NULL	

초기 데이터 계획

- 파트 B 독립 개발용 더미 데이터: 파트 A의 가게 등록 API를 기다리지 않도록, DataInitRunner에 가게 1번(루키즈 베이커리)을 미리 심어두고 메뉴판·장바구니 화면을 개발합니다.
- 파트 C 독립 개발용 더미 데이터: 파트 B의 주문 생성 API를 기다리지 않도록, DataInitRunner에 5001번 주문(바닐라라떼 2잔, 11,000원, 접수대기 상태)을 미리 심어두고 대시보드 화면을 개발합니다.
- 더미 데이터 스키마 합의: 위 더미 데이터의 컬럼 구성은 파트 A/B/C가 초반에 한 번 맞춰보아야 합니다. 특히 orders 테이블은 B와 C가 같이 참조하므로, 두 파트가 만드는 더미 Order 엔티티의 필드명·타입이 다르면 나중에 병합할 때 충돌이 납니다.
- 매장 카테고리 값: 프론트/백엔드가 공유하는 고정 목록(예: 베이커리, 카페, 분식, 편의점, 기타)을 시드로 넣어 등록 화면 드롭다운에 사용합니다.
- 시딩 방식: Spring Boot의 CommandLineRunner(DataInitRunner)를 사용합니다. 개발 초기에는 이 방식이 가장 간단합니다.